

Rapport de Stage

Infrastructure as Code & Automation pilotée par Chatbot Intelligent

Étudiant : [Votre Nom]

Année universitaire 2025-2026

Table des matières

1	Remerciements	3
2	Résumé	3
3	Introduction générale	4
3.1	Contexte	4
3.2	Problématique	4
3.3	Solution proposée	4
3.4	Méthodologie de travail	4
4	Chapitre 1 — Étude préliminaire et spécification des besoins	5
4.1	1.1 Présentation du projet	5
4.2	1.2 Objectifs	5
4.3	1.3 Identification des acteurs	5
4.4	1.4 Besoins fonctionnels	5
4.5	1.5 Besoins non fonctionnels	6
5	Chapitre 2 — Étude technique et choix technologiques	7
5.1	2.1 Frontend : Angular	7
5.2	2.2 Backend : Spring Boot 3.2 (Java 21)	7
5.3	2.3 Intelligence artificielle : Ollama + Llama 3	7
5.4	2.4 Génération IaC : Thymeleaf (mode TEXT)	7
5.5	2.5 Exécution des déploiements	7
5.5.1	VMs : VirtualBox	7
5.5.2	Conteneurs : MicroShift (OpenShift)	7
5.6	2.6 Base de données	8
5.7	2.7 Conteneurisation : Docker Compose	8
6	Chapitre 3 — Conception	9
6.1	3.1 Architecture en couches	9
6.2	3.2 Diagramme de cas d'utilisation	9
6.3	3.3 Diagramme de séquence — Déploiement d'une VM	9
6.4	3.4 Modèle de données	9
6.5	3.5 Pipeline de déploiement	9
7	Chapitre 4 — Réalisation	11
7.1	4.1 Backend Spring Boot	11
7.2	4.2 Frontend Angular	11
7.3	4.3 Mise en place de l'infrastructure réelle	11
7.3.1	Cluster OpenShift local	11
7.3.2	VMs VirtualBox	11

7.4	4.4 Résultats obtenus	11
8	Chapitre 5 — Tests et validation	12
8.1	5.1 Tests automatisés	12
8.2	5.2 Tests de bout en bout (réels)	12
9	Chapitre 6 — Difficultés rencontrées et solutions	12
10	Conclusion générale et perspectives	12
10.1	Bilan	12
10.2	Compétences acquises	13
10.3	Perspectives	13
11	Annexes	13
11.1	A. API principale	13
11.2	B. Démarrage de la plateforme	13
11.3	C. Structure des répertoires	13

1 Remerciements

Je tiens à remercier mon encadrant pédagogique et mon tuteur en entreprise pour leur accompagnement, leurs conseils et leur disponibilité tout au long de ce stage. Je remercie également l'ensemble de l'équipe pour son accueil et sa confiance.

2 Résumé

Ce rapport présente la conception et la réalisation d'une plateforme d'automatisation d'infrastructure pilotée par un chatbot intelligent. La plateforme permet de créer, configurer et gérer des machines virtuelles et des conteneurs à partir de demandes exprimées en langage naturel (français ou anglais), sans expertise technique préalable.

La solution repose sur une architecture moderne : frontend Angular, backend Spring Boot sécurisé par JWT, intelligence artificielle 100 % locale (Ollama + Llama 3), génération de code Infrastructure as Code via des templates Thymeleaf (Terraform, YAML OpenShift, KubeVirt), et un pipeline de déploiement réel exécutant les ressources sur VirtualBox (machines virtuelles) et sur un cluster OpenShift local (MicroShift).

Mots-clés : Infrastructure as Code, Chatbot, IA locale, Spring Boot, Angular, Terraform, OpenShift, VirtualBox, DevOps.

3 Introduction générale

3.1 Contexte

L’Infrastructure as Code (IaC) est devenue une pratique incontournable du DevOps : l’infrastructure est décrite par du code, versionnée, reproductible et automatisable. Cependant, ces outils (Terraform, Kubernetes, OpenShift, vSphere) exigent une expertise technique que ne possèdent pas tous les acteurs d’une entreprise.

En parallèle, les modèles de langage (LLM) ouvrent la voie à des interfaces en langage naturel capables de traduire une intention humaine en actions techniques.

3.2 Problématique

Comment permettre à un utilisateur non expert de provisionner de l’infrastructure (VMs, conteneurs) de manière fiable, sécurisée et traçable, sans maîtriser les outils d’automatisation sous-jacents ?

3.3 Solution proposée

Une plateforme web dont le point d’entrée unique est un **chatbot intelligent** capable de :

- comprendre une demande en langage naturel (français/anglais) ;
- identifier le type de ressource (VM ou conteneur) et la plateforme cible ;
- extraire les paramètres techniques (CPU, RAM, stockage, image, réseau) ;
- générer dynamiquement le code IaC approprié ;
- valider et exécuter le déploiement réel de la ressource ;
- journaliser l’ensemble des opérations (traçabilité).

3.4 Méthodologie de travail

Le projet a été mené en mode itératif sur 4 semaines : analyse, conception, développement backend, développement frontend, tests et validation, documentation.

4 Chapitre 1 — Étude préliminaire et spécification des besoins

4.1 1.1 Présentation du projet

Le projet s'inscrit dans une démarche d'automatisation de l'infrastructure de l'entreprise. Deux environnements d'exécution sont ciblés par le document officiel :

- **VMware vSphere** : provisioning et gestion de machines virtuelles ;
- **OpenShift Container Platform** : déploiement de conteneurs Kubernetes et de machines virtuelles via OpenShift Virtualization (KubeVirt).

4.2 1.2 Objectifs

- Automatiser le provisioning d'infrastructure ;
- Supporter machines virtuelles et conteneurs ;
- Unifier la gestion VMware et OpenShift derrière une interface unique ;
- Simplifier les opérations IT via le langage naturel ;
- Introduire l'IA dans l'automatisation cloud ;
- Garantir standardisation, traçabilité et fiabilité ;
- Coût total de la solution : 0 € (open-source uniquement).

4.3 1.3 Identification des acteurs

Acteur	Description	Privilèges
Utilisateur	Développeur, ops ou chef de projet	S'authentifier, demander un déploiement, visualiser le code, consulter l'historique, annuler
Administrateur	Responsable infrastructure	Tout + valider les demandes, gérer les utilisateurs et les quotas
Moteur de déploiement	Composant backend (RealDeployExecutor)	Exécuter les déploiements réels, journaliser

4.4 1.4 Besoins fonctionnels

Réf.	Cas d'utilisation	Description
UC-01	S'authentifier	Connexion JWT (login/mot de passe)
UC-02	S'inscrire	Création de compte
UC-04	Demander un déploiement	Message en langage naturel → code IaC
UC-05	Visualiser le code généré	Affichage du Terraform/YAML produit
UC-06	Confirmer le déploiement	Exécution réelle du code généré
UC-07	Clarification	Rejet des combinaisons invalides (ex. conteneur sur vSphere)
UC-08	Consulter l'historique	Liste des demandes passées
UC-09	Dashboard	Statistiques et suivi
UC-10	Suivi temps réel	Progression du déploiement via WebSocket
UC-11	Gérer les utilisateurs	Administration (ADMIN)
UC-12	Valider les demandes	Workflow d'approbation (ADMIN)
UC-13	Quotas	Limitation des ressources par utilisateur

4.5 1.5 Besoins non fonctionnels

- **Sécurité** : JWT, hashage BCrypt, aucun secret en dur, rôles ;
- **Performance** : génération du code IaC < 500 ms ;
- **Fiabilité** : journalisation complète, gestion d'erreurs standardisée ;
- **Maintenabilité** : architecture en couches, tests automatisés, documentation Swagger ;
- **Confidentialité** : IA 100 % locale, aucune donnée transmise à l'extérieur ;
- **Coût** : 0 € — uniquement des composants open-source.

5 Chapitre 2 — Étude technique et choix technologiques

5.1 2.1 Frontend : Angular

Critère	Justification
Structure	Framework complet et cadré (standard entreprise)
Langage	TypeScript : typage fort, moins d'erreurs
Réactivité	RxJS + WebSocket pour le chat temps réel
Écosystème	Composants, routing, guards, interceptors intégrés

Alternative écartée : React — plus libre mais moins structurant pour un projet d'entreprise de cette taille.

5.2 2.2 Backend : Spring Boot 3.2 (Java 21)

- Standard de l'industrie pour les API d'entreprise ;
- **Spring Security** + **JWT** : authentification stateless ;
- **Spring Data JPA** : persistance (H2 en dev, MySQL en prod) ;
- **Spring AI** : intégration native d'Ollama ;
- **Thymeleaf** : génération de texte (templates IaC) ;
- Tests : JUnit 5 + Mockito.

5.3 2.3 Intelligence artificielle : Ollama + Llama 3

Critère	Ollama + Llama 3	OpenAI/ChatGPT
Coût	0 €	20-50 \$/mois
Confidentialité	100 % local, offline	Données envoyées au cloud
Quotas	Aucun	Limites API
Latence réseau	Aucune (local)	Dépendante d'internet

Le modèle llama3 extrait les paramètres de la demande et les retourne en JSON structuré (type de ressource, plateforme, CPU, RAM, stockage, image, réseau).

5.4 2.4 Génération IaC : Thymeleaf (mode TEXT)

Thymeleaf, moteur de templates intégré à Spring Boot, est détourné de son usage HTML classique pour générer du code : Terraform (vSphere), YAML OpenShift (Deployment, Service, Route) et YAML KubeVirt (VM sur OpenShift). Les templates sont versionnés, ce qui garantit standardisation et reproductibilité.

5.5 2.5 Exécution des déploiements

5.5.1 VMs : VirtualBox

Le document officiel cible VMware vSphere. Un serveur vCenter ne pouvant pas s'installer sur un PC personnel, l'exécution réelle locale est assurée par **VirtualBox** (gratuit, open-source) piloté par l'outil en ligne de commande **VBoxManage** : le chatbot crée réellement la VM avec les CPU/RAM/disque demandés. Le code Terraform vSphere généré reste prêt pour une infrastructure d'entreprise.

5.5.2 Conteneurs : MicroShift (OpenShift)

OpenShift Local (CRC) exige Hyper-V, absent de Windows 11 Famille. La solution retenue : **MicroShift 4.18**, la distribution OpenShift officielle allégée de Red Hat, installée dans une VM VirtualBox AlmaLinux 9. C'est un **vrai OpenShift** : même API, commande `oc`, Routes, conteneurs réels.

5.6 2.6 Base de données

- **H2** (développement) : en mémoire, zéro installation ;
- **MySQL Community 8.0** (production) : gratuit, fiable, déployé via Docker Compose.

5.7 2.7 Conteneurisation : Docker Compose

La stack complète (backend, frontend nginx, MySQL) est décrite dans un `docker-compose.yml` : déploiement reproductible en une commande.

6 Chapitre 3 — Conception

6.1 3.1 Architecture en couches

Couche	Composants	Responsabilité
Présentation	Angular 17+, RxJS, WebSocket	Interface chatbot, dashboard, admin
API	Spring Security, JWT Filter, CORS	Authentification, autorisation
Application	Controllers, Services Spring Boot	Logique métier, orchestration
Intelligence	Ollama (Llama 3), Thymeleaf	NLP local, extraction, génération IaC
Données	Spring Data JPA, H2 / MySQL	Persistance des demandes et logs
Exécution	RealDeployExecutor (VBoxManage, oc)	Pipeline de déploiement réel
Infrastructure	VirtualBox, MicroShift/OpenShift	VMs et conteneurs réels

6.2 3.2 Diagramme de cas d'utilisation

L'utilisateur interagit avec le chatbot (UC-04), visualise le code généré (UC-05), confirme le déploiement (UC-06), consulte l'historique (UC-08) et le dashboard (UC-09). L'administrateur gère les utilisateurs (UC-11), valide les demandes (UC-12) et configure les quotas (UC-13). La confirmation d'un déploiement déclenche le pipeline d'exécution (VALIDATION → PLAN → APPLY → VERIFY).

6.3 3.3 Diagramme de séquence — Déploiement d'une VM

1. L'utilisateur écrit « Je veux une VM Ubuntu 2 CPU, 4 Go RAM » ;
2. Le frontend envoie le message au backend (POST /api/chatbot/process, JWT) ;
3. LlmService interroge Ollama (local) et extrait les paramètres JSON ;
4. IaCGeneratorService génère le code via le template Thymeleaf ;
5. La demande et le code sont persistés (H2/MySQL) ;
6. L'utilisateur confirme (POST /api/deploy/{id}) ;
7. DeployService délègue à RealDeployExecutor qui exécute réellement VBoxManage (création VM) ou oc apply (OpenShift) ;
8. Chaque étape est journalisée (DeploymentLog) et poussée en WebSocket ;
9. La ressource est créée ; une notification de succès est émise.

6.4 3.4 Modèle de données

- **User** : id, username, email, password (BCrypt), rôle (USER/ADMIN) ;
- **InfrastructureRequest** : id, userMessage, resourceType (VM/CONTAINER), extractedParams (JSON), generatedCode, targetPlatform (VSPHERE/OPENSIFT), status, dates, errorMessage, utilisateur propriétaire ;
- **DeploymentLog** : id, demande associée, étape, niveau, message, timestamp.

6.5 3.5 Pipeline de déploiement

Étape	VM (VirtualBox)	OpenShift (MicroShift)
VALIDATION	VBoxManage --version + contrôle doublon	oc apply --dry-run=server
PLAN	Résumé des ressources à créer	Liste des ressources (-o name)
APPLY	createvm, modifyvm, createmedium, storagectl, storageattach	oc apply -f manifest.yaml

Étape	VM (VirtualBox)	OpenShift (MicroShift)
VERIFY	<code>showvminfo</code> (état de la VM)	<code>oc get</code> (ressources créées)
Annulation	<code>unregistervm --delete</code>	<code>oc delete -f manifest.yaml</code>

En cas d'échec : statut FAILED, message d'erreur persisté, notification temps réel.

7 Chapitre 4 — Réalisation

7.1 4.1 Backend Spring Boot

Structure en packages : **controller** (API REST), **service** (métier), **repository** (JPA), **model** (entités), **security** (JWT), **dto**, **exception**, **config**.

Services principaux :

- **AuthService / UserService** : inscription, connexion, gestion des utilisateurs ;
- **LlmService** : dialogue avec Ollama, extraction JSON des paramètres ;
- **IaCGeneratorService** : rendu des templates Thymeleaf ;
- **DeployService** : orchestration du déploiement (statuts, logs, notifications) ;
- **RealDeployExecutor** : exécution réelle (ProcessBuilder) de VBoxManage et oc ;
- **QuotaService** : limitation des ressources par utilisateur ;
- **ProgressNotificationService** : progression temps réel via WebSocket STOMP.

7.2 4.2 Frontend Angular

Modules : **auth** (connexion/inscription), **chat** (interface conversationnelle avec affichage du code et bouton Déployer), **history** (historique des demandes), **admin** (gestion utilisateurs). Guards de routes, intercepteur HTTP (JWT automatique), client WebSocket STOMP.

7.3 4.3 Mise en place de l'infrastructure réelle

7.3.1 Cluster OpenShift local

1. Création d'une VM VirtualBox **microshift** (2 vCPU, 4 Go RAM, 40 Go) ;
2. Installation automatisée d'AlmaLinux 9 via image cloud + cloud-init (seed ISO) ;
3. Installation de MicroShift 4.18 (dépôts officiels mirror.openshift.com) ;
4. Configuration du pull secret Red Hat (compte développeur gratuit) ;
5. Redirections NAT : API 16443→6443, Routes 9080→80 / 9443→443, SSH 2222→22 ;
6. Création d'un compte de service et d'un token pour le backend.

7.3.2 VMs VirtualBox

Le backend exécute VBoxManage sur la machine hôte : création de la VM (**iac-vm-<id>**), configuration CPU/RAM, création et attachement du disque VDI. L'annulation supprime réellement la VM.

7.4 4.4 Résultats obtenus

- Demande « VM Ubuntu 2 CPU / 4 Go / 20 Go » → VM réelle créée dans VirtualBox (ostype Ubuntu 64-bit, specs exactes) ;
- Demande « nginx 3 replicas sur OpenShift » → Deployment + Service + Route réels, 3 pods Running, application accessible via la Route (HTTP 200) ;
- Historique et logs de déploiement persistés et consultables ;
- Annulation réelle (VM supprimée, ressources OpenShift supprimées).

8 Chapitre 5 — Tests et validation

8.1 5.1 Tests automatisés

59 tests unitaires (JUnit 5 + Mockito), 100 % au vert :

Classe de test	Couverture
AuthServiceTest	Inscription, connexion, JWT
UserServiceTest	Gestion des utilisateurs
IaCGeneratorServiceTest	Génération Terraform / OpenShift / KubeVirt
LlmServiceTest	Extraction des paramètres, erreurs de parsing
DeployServiceTest	Statuts, refus (doublon, approbation), annulation
RealDeployExecutorTest	Mapping OS, nommage, gestion d'erreur d'exécution
QuotaServiceTest	Quotas par utilisateur
ProgressNotificationServiceTest	Notifications WebSocket
ChatbotControllerTest	Endpoints de l'API

8.2 5.2 Tests de bout en bout (réels)

Scénario	Résultat
Chat → génération Terraform vSphere	code généré avec paramètres extraits
Chat → VM réelle VirtualBox	VM visible dans VirtualBox (specs exactes)
Chat → conteneurs réels OpenShift	Pods Running, Route HTTP 200
Annulation	VM / ressources supprimées réellement
Historique	demandes et logs persistés

9 Chapitre 6 — Difficultés rencontrées et solutions

Difficulté	Solution adoptée
Pas de serveur vCenter sur PC personnel	VirtualBox + VBoxManage pour l'exécution réelle des VMs
Windows 11 Famille : Hyper-V absent → OpenShift Local impossible	MicroShift (OpenShift officiel allégé) dans une VM VirtualBox
RAM limitée (16 Go) entre Ollama, la VM et les outils	Optimisation : requests/limites modestes dans les manifests, gestion mémoire
Ressources des pods trop gourmandes → échec d'ordonnancement	Template YAML corrigé (requests 100m/128Mi, limits = paramètres)
Miroirs CentOS inaccessibles	AlmaLinux 9 (compatible MicroShift) via image cloud + cloud-init
Installation OS automatisée non supportée par VirtualBox (EL9)	Image cloud GenericCloud + seed ISO cloud-init générée via Docker

10 Conclusion générale et perspectives

10.1 Bilan

L'ensemble des objectifs du cahier des charges est atteint : une plateforme web fonctionnelle transforme une demande en langage naturel en infrastructure réelle — VMs créées dans VirtualBox et conteneurs déployés sur un vrai cluster OpenShift — avec génération de code IaC standardisé, traçabilité complète, sécurité JWT et coût nul.

10.2 Compétences acquises

- Développement full-stack (Angular, Spring Boot) ;
- Intégration d'un LLM local dans une application d'entreprise ;
- Infrastructure as Code (Terraform, manifests Kubernetes/OpenShift) ;
- Virtualisation (VirtualBox) et conteneurisation (Docker, OpenShift/MicroShift) ;
- Sécurité applicative (JWT, rôles) et tests automatisés.

10.3 Perspectives

- Connexion à un vrai vCenter / cluster OpenShift d'entreprise (simple configuration) ;
- KubeVirt réel (VMs dans OpenShift) sur un cluster plus puissant ;
- Installation automatique de l'OS dans les VMs créées (Ubuntu autoinstall) ;
- Notifications email/Discord en production ;
- Déploiement de la plateforme elle-même sur Kubernetes.

11 Annexes

11.1 A. API principale

Endpoint	Description
POST /api/auth/register / login	Authentification JWT
POST /api/chatbot/process	Message → code IaC
GET /api/chatbot/requests	Historique des demandes
POST /api/deploy/{id}	Déployer (réel)
GET /api/deploy/{id}/status	Statut + logs
DELETE /api/deploy/{id}	Annuler (suppression réelle)
GET /api/users (ADMIN)	Gestion des utilisateurs
GET /swagger-ui.html	Documentation interactive

11.2 B. Démarrage de la plateforme

1. IA locale

```
ollama serve
```

2. Cluster OpenShift

```
VBoxManage startvm microshift --type headless
```

3. Backend (mode réel)

```
cd iac-chatbot-backend
```

```
mvn spring-boot:run
```

4. Frontend

```
cd iac-chatbot-angular
```

```
npx ng serve # http://localhost:4200
```

Comptes : admin / password123 (ADMIN), user / password123 (USER).

11.3 C. Structure des répertoires

iac-chatbot-backend/	API Spring Boot (Java 21)
iac-chatbot-angular/	Frontend Angular 17+
openshift/	Cluster MicroShift (oc.exe, token, pull secret, seed)
docker-compose.yml	Stack prod (backend, frontend, MySQL)